MINI-PROJETS TP algorithmique avancée

Niveau : Master 1 — ASR / GL

THÈME 1 — Développement d'un détecteur de similarité entre codes sources

Objectif général : développer une application capable de comparer un code source de référence à plusieurs codes sources candidats, de calculer un score de similarité pour chacun et de classer les résultats du plus similaire au moins similaire.

Le programme doit permettre de visualiser les scores obtenus et, si possible, de mettre en évidence les fragments de code similaires.

Niveau 1 — Facile : comparaison textuelle

Développer un programme qui compare des codes sources écrits dans le même langage de programmation, avec les mêmes noms de variables et le même ordre des instructions et des déclarations. Le programme doit calculer un score de similarité pour chaque fichier candidat et présenter les résultats sous forme de classement.

Niveau 2 — Moyen : comparaison avec transformations du code

Reprendre les fonctionnalités du niveau 1 et permettre la détection de similarités malgré des modifications telles que le changement des noms de variables ou le déplacement de fonctions, de déclarations de variables et de blocs d'instructions dans le fichier source. Les programmes restent écrits dans le même langage de programmation.

Niveau 3 — Difficile : comparaison entre langages différents

Reprendre les fonctionnalités du niveau 2 et permettre la comparaison de codes sources écrits dans des langages de programmation différents, lorsque ces codes mettent en œuvre des algorithmes ou des traitements similaires. Le programme doit dépasser la simple comparaison textuelle pour identifier des ressemblances structurelles ou algorithmiques.

Les étudiants doivent expliquer la méthode utilisée pour calculer les scores et présenter ses limites. Ils doivent notamment tester leur solution sur des codes identiques, modifiés et sans relation apparente. Un score de similarité ne constitue pas, à lui seul, une preuve de plagiat.

THÈME 2 — Navigation intelligente d'un robot dans un environnement dynamique

Objectif général : développer une application permettant à un robot de se déplacer dans une grille depuis un point de départ jusqu'à une destination en évitant les obstacles. L'application doit comporter une interface graphique affichant la grille, le robot, la destination, les obstacles et le chemin calculé.

Le chemin doit être représenté graphiquement par une ligne reliant le robot à la destination et contournant les obstacles.

Niveau 1 — Facile : recherche d'un chemin

Développer un programme permettant au robot de trouver un chemin valide entre son point de départ et la destination dans un environnement comportant des obstacles fixes. Le chemin calculé doit être affiché graphiquement sous la forme d'une ligne reliant le robot à la destination.

Niveau 2 — Moyen : gestion des obstacles en temps réel

Reprendre les fonctionnalités du niveau 1 et permettre l'apparition ou la disparition d'obstacles pendant le déplacement du robot. Lorsque le chemin prévu devient impraticable, le programme doit rechercher un nouvel itinéraire et mettre à jour la ligne affichée afin de relier le robot à la destination en évitant les obstacles.

Niveau 3 — Difficile : recherche du plus court chemin sous contrainte de temps

Reprendre les fonctionnalités du niveau 2 et optimiser la navigation afin de trouver le plus court chemin vers la destination, dans un temps de calcul borné et dans un environnement connu. Lorsqu'un obstacle apparaît ou disparaît, le programme doit recalculer un itinéraire optimal si nécessaire et actualiser son affichage.

Les étudiants doivent expliquer les algorithmes utilisés, justifier leurs choix et, si possible, comparer les performances de plusieurs méthodes en fonction du temps de calcul, de la longueur du chemin et du nombre de cases explorées.

THÈME 3 — Développement d'un joueur intelligent de Gomoku (Puissance 5)

Objectif général : développer une application permettant de jouer au Gomoku, un jeu dans lequel le premier joueur qui aligne cinq pierres de sa couleur horizontalement, verticalement ou en diagonale remporte la partie.

Niveau 1 — Facile : joueur automatique contre un humain

Développer une application avec une interface graphique permettant à un humain de jouer contre un programme. Le programme doit élaborer une stratégie de jeu, choisir ses coups en empêchant l'adversaire d'aligner ses pierres et détecter les situations de victoire ou de défaite.

Niveau 2 — Moyen : compétition entre programmes sur un réseau

Reprendre les fonctionnalités du niveau 1 et permettre à deux programmes développés par des binômes différents de s'affronter.

Les étudiants doivent concevoir et mettre en œuvre une plateforme commune de jeu en réseau permettant aux programmes de communiquer, d'échanger les coups et de respecter les règles de la partie. Le protocole de communication doit être commun à tous les binômes afin de garantir l'interopérabilité de leurs programmes.

Chaque binôme développe ensuite sa propre stratégie et peut la confronter à celle des autres binômes.

Niveau 3 — Difficile : stratégie optimisée sous contrainte de temps

Reprendre les fonctionnalités du niveau 2 et imposer un temps maximal de réflexion pour chaque coup, aussi bien lors d'une partie contre un humain que lors d'une partie entre deux programmes.

Chaque binôme doit optimiser sa stratégie afin de choisir des coups pertinents dans le délai imparti. Il doit maîtriser la complexité de la recherche et garantir qu'un coup soit fourni avant l'expiration du délai.

Les étudiants doivent mesurer les temps de calcul et expliquer les techniques d'optimisation utilisées.

CONDITIONS GÉNÉRALES COMMUNES AUX TROIS THÈMES

1. Langage de programmation et interface graphique

- **Langage de programmation :** libre choix des étudiants.
- **Interface graphique :** obligatoire pour tous les thèmes et tous les niveaux. Elle doit permettre d'utiliser les fonctionnalités du projet et de visualiser les résultats de manière claire.
- Le code source doit être fourni et le programme doit pouvoir être exécuté et testé lors de la présentation.

2. Travail personnel et originalité

Chaque groupe, composé d'au plus trois étudiants, doit développer sa propre solution et être capable d'expliquer et de justifier ses choix algorithmiques et techniques.

Le plagiat, la copie de solutions existantes et le recours aux outils d'intelligence artificielle générative pour produire le code source ou le rapport sont interdits.

Les étudiants doivent maîtriser leur travail et être capables d'expliquer leur implémentation, de justifier leurs choix et de répondre aux questions lors de la présentation. Des modifications du code source peuvent leur être demandés pendant la démonstration afin de vérifier leur compréhension.

3. Modalités d'évaluation

La notation est progressive selon le niveau de difficulté réalisé :

- **Niveau 1 :** note de base sur 13 points.
- **Niveau 2 :** 3 points supplémentaires.
- **Niveau 3 :** 4 points supplémentaires, cumulables avec les points du niveau 2.

La note finale est plafonnée à **20/20**.

Les points sont accordés en fonction de la réalisation effective des fonctionnalités du niveau concerné, de leur bon fonctionnement et de leur démonstration.

Pour obtenir les points d'un niveau supérieur, le binôme doit également satisfaire aux exigences des niveaux précédents.

4. Rapport technique

Pour chaque thème, le binôme doit fournir un rapport électronique de **trois pages maximum** présentant notamment :

- la stratégie et les algorithmes utilisés ;

- les choix de conception et les structures de données retenues ;
- les optimisations éventuellement réalisées ;
- les résultats des tests ;
- les limites de la solution développée.

Le rapport doit être rédigé par les étudiants et refléter leur travail personnel.

5. Date limite de remise

Le projet, son code source, les rapports techniques correspondant aux niveaux réalisés et les éléments nécessaires à son exécution doivent être remis **au plus tard le 10 décembre 2026**.

Chaque groupe doit être en mesure de présenter son application, de démontrer son fonctionnement et de justifier les choix effectués.